

Operating System

Assignment #1

학과: 컴퓨터정보공학부

담당교수: 교수님

학번: 2024402055

이름: 박현지

1. Introduction

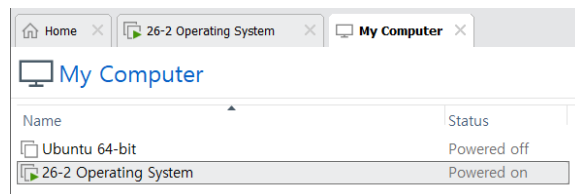
본 과제는 Linux 운영체제 환경을 직접 구성하고, Linux Kernel 소스 코드를 컴파일 및 수정하는 과정을 실습하는 것을 목적으로 한다. 먼저 Ubuntu 환경을 준비하고 과제에서 요구하는 사용자 계정과 개발 환경을 설정하였다. 이후 Linux Kernel 5.4.282 소스 코드를 이용하여 커널을 직접 컴파일하고 설치한 뒤, 수정한 커널로 정상적으로 부팅되는지 확인하였다. 또한 Ctags와 Cscope를 활용하여 커널 소스 코드에서 특정 함수와 실행 위치를 탐색하고, `printk()`를 이용해 Kernel Message가 출력되도록 소스 코드를 수정하였다. 마지막으로 `dmesg`와 `grep`을 통해 수정한 메시지가 정상적으로 출력되는지 확인함으로써 Linux Kernel의 빌드, 초기화 흐름, 소스 탐색 및 디버깅 과정을 실습하였다.

2. Result

A. Assignment 1-1

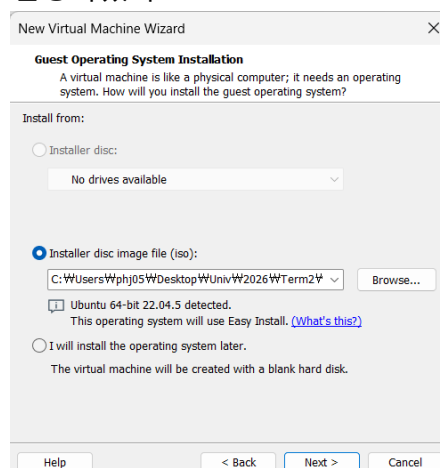
i. 가상 머신 생성

Ubuntu Linux를 설치하기 위해 VMware에서 새로운 가상 머신을 생성하였다.



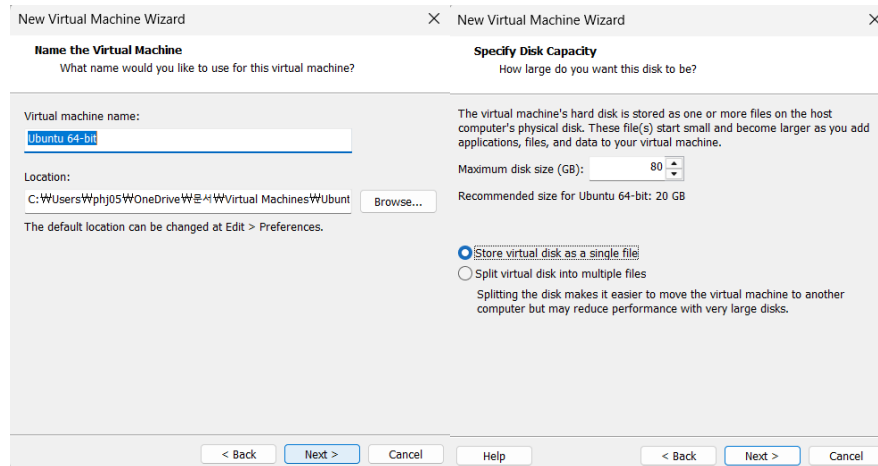
ii. Ubuntu ISO 설정

Ubuntu 20.04.6 Desktop 64bit ISO 파일을 가상 머신의 설치 이미지로 설정하였다.



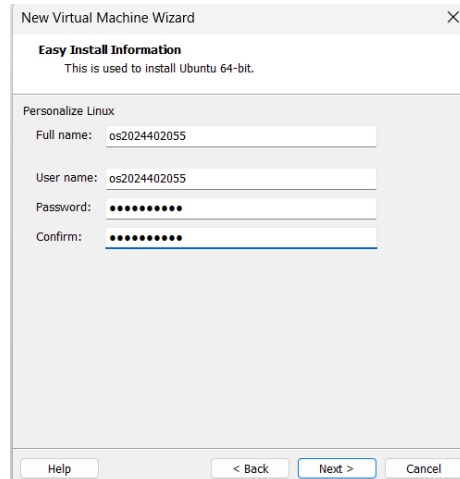
iii. Ubuntu 설치

가상 머신을 실행한 후 Ubuntu 설치 프로그램을 통해 운영체제 설치를 진행하였다.



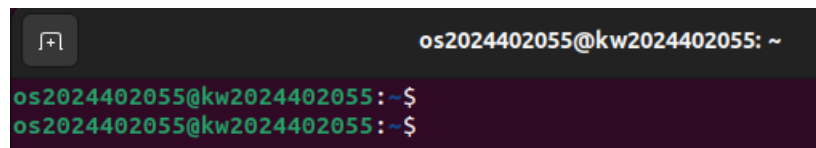
iv. 사용자 계정 생성

과제 요구사항에 따라 사용자 계정 ID를 os2024402055 형식으로 설정하였다.



v. 설치 완료

설치 완료 후 시스템을 재부팅하고 터미널을 실행하여 정상적으로 Ubuntu가 실행되는 것을 확인하였다.



B. Assignment 1-2: Linux Kernel Compile

Linux Kernel 5.4.282를 다운로드한 뒤, 학번을 포함한 커널 버전을 생성하도록 설정하고 커널 컴파일 및 설치를 진행하였다. 최종적으로 새로 컴파일한 커널로 부팅한 후 `uname -r` 명령어를 통해 커널 버전을 확인하였다.

i. 커널 버전 설정

커널 소스의 Makefile을 vi 편집기로 열어 EXTRAVERSION 항목을 다음과 같이 수정하였다.

[illegible]

-j4 옵션은 최대 4개의 작업을 병렬로 실행하여 커널을 컴파일한다.

```
LD [M] sound/xen/snd_xen_front.ko
LD [M] virt/lib/irqbypass.ko
os2024402055@kw2024402055:~/26-2_OS/Assignment1/linux-5.4.282$ echo $?
0
os2024402055@kw2024402055:~/26-2_OS/Assignment1/linux-5.4.282$ make kernelrelease
5.4.282-os2024402055
os2024402055@kw2024402055:~/26-2_OS/Assignment1/linux-5.4.282$ ls -lh arch/x86/boot/bzImage
-rw-rw-r-- 1 os2024402055 os2024402055 11M 9월 18 05:46 arch/x86/boot/bzImage
```

컴파일 완료 후 echo \$? 명령어를 이용하여 정상 종료 여부를 확인하였다.

실행 결과 0이 출력되어 컴파일이 정상적으로 완료되었음을 확인하였다. 또한 생성된 커널 이미지와 커널 버전을 그 다음 명령어로 확인하였다.

실행 결과 5.4.282-os2024402055 버전의 커널이 생성되었으며, arch/x86/boot/bzImage 파일이 정상적으로 생성된 것을 확인하였다.

iv. 커널 모듈 설치

컴파일된 커널에서 사용하는 모듈을 시스템에 설치하기 위해 다음 명령어를 실행하였다.

```
os2024402055@kw2024402055:~/26-2_OS/Assignment1/linux-5.4.282$ sudo make modules_install
[sudo] password for os2024402055:
INSTALL arch/x86/crypto/aegis128-aesni.ko
INSTALL arch/x86/crypto/aesni-intel.ko
INSTALL arch/x86/crypto/blowfish-x86_64.ko
INSTALL arch/x86/crypto/camellia-aesni-avx-x86_64.ko
INSTALL arch/x86/crypto/camellia-aesni-avx2.ko
INSTALL arch/x86/crypto/camellia-x86_64.ko
INSTALL arch/x86/crypto/cast5-avx-x86_64.ko
INSTALL arch/x86/crypto/cast6-avx-x86_64.ko
INSTALL arch/x86/crypto/chacha-x86_64.ko
INSTALL arch/x86/crypto/crc32-pclmul.ko
INSTALL arch/x86/crypto/crc32-pclmul.ko
```

컴파일된 커널 모듈을 커널 버전별 디렉토리에 설치한다.

설치 완료 후 echo \$?를 사용하여 정상적으로 완료되었음을 확인하였다.

```
INSTALL sound/usb/usx2y/snd-usb-us122l.ko
INSTALL sound/usb/usx2y/snd-usb-usx2y.ko
INSTALL sound/x86/snd-hdmi-lpe-audio.ko
INSTALL sound/xen/snd_xen_front.ko
INSTALL virt/lib/irqbypass.ko
DEPMOD 5.4.282-os2024402055
os2024402055@kw2024402055:~/26-2_OS/Assignment1/linux-5.4.282$ echo $?
0
```

v. 커널 설치

컴파일된 커널 이미지를 /boot 디렉토리에 설치하기 위해 다음 명령어를 실행하였다.

```

os2024402055@kw2024402055:~/26-2_OS/Assignment1/linux-5.4.282$ sudo make install
sh ./arch/x86/boot/install.sh 5.4.282-os2024402055 arch/x86/boot/bzImage \
    System.map "/boot"
run-parts: executing /etc/kernel/postinst.d/initramfs-tools 5.4.282-os2024402055 /boot/vmlinuz-5.4.282-os2024402055
update-initramfs: Generating /boot/initrd.img-5.4.282-os2024402055
run-parts: executing /etc/kernel/postinst.d/unattended-upgrades 5.4.282-os2024402055 /boot/vmlinuz-5.4.282-os2024402055
run-parts: executing /etc/kernel/postinst.d/update-notifier 5.4.282-os2024402055 /boot/vmlinuz-5.4.282-os2024402055
run-parts: executing /etc/kernel/postinst.d/xx-update-initrd-links 5.4.282-os2024402055 /boot/vmlinuz-5.4.282-os2024402055
I: /boot/initrd.img.old is now a symlink to initrd.img-6.8.0-138-generic
I: /boot/initrd.img is now a symlink to initrd.img-5.4.282-os2024402055
run-parts: executing /etc/kernel/postinst.d/zz-shim 5.4.282-os2024402055 /boot/vmlinuz-5.4.282-os2024402055
run-parts: executing /etc/kernel/postinst.d/zz-update-grub 5.4.282-os2024402055 /boot/vmlinuz-5.4.282-os2024402055
Sourcing file '/etc/default/grub'
Sourcing file '/etc/default/grub.d/init-select.cfg'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.8.0-138-generic
Found initrd image: /boot/initrd.img-6.8.0-138-generic
Found linux image: /boot/vmlinuz-6.8.0-111-generic
Found initrd image: /boot/initrd.img-6.8.0-111-generic
Found linux image: /boot/vmlinuz-5.4.282-os2024402055
Found initrd image: /boot/initrd.img-5.4.282-os2024402055
Found memtest86+ image: /boot/memtest86+.elf
Found memtest86+ image: /boot/memtest86+.bin
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
done
os2024402055@kw2024402055:~/26-2_OS/Assignment1/linux-5.4.282$ echo $?
0

```

설치 과정에서 update-initramfs가 실행되어 새 커널에서 부팅할 때 필요한 초기 RAM 파일 시스템인 initrd.img가 생성되었다.

확인 결과 다음과 같이 새 커널 관련 파일들이 정상적으로 생성되었다.

```

os2024402055@kw2024402055:~/26-2_OS/Assignment1/linux-5.4.282$ ls -lh /boot | grep 5.4.282
-rw-r--r-- 1 root root 229K 9월 18 06:34 config-5.4.282-os2024402055
lrwxrwxrwx 1 root root 31 9월 18 06:35 initrd.img -> initrd.img-5.4.282-os2024402055
-rw-r--r-- 1 root root 350M 9월 18 06:35 initrd.img-5.4.282-os2024402055
-rw-r--r-- 1 root root 4.8M 9월 18 06:34 System.map-5.4.282-os2024402055
lrwxrwxrwx 1 root root 28 9월 18 06:34 vmlinuz -> vmlinuz-5.4.282-os2024402055
-rw-r--r-- 1 root root 11M 9월 18 06:34 vmlinuz-5.4.282-os2024402055

```

vi. GRUB에 새 커널 등록

새롭게 설치한 커널을 부팅 시 선택할 수 있도록 GRUB 설정을 갱신하였다. 이후 GRUB 설정 파일을 검색하여 Linux Kernel 5.4.282가 정상적으로 등록되었는지 확인하였다.

```

os2024402055@kw2024402055:~/26-2_OS/Assignment1/linux-5.4.282$ sudo update-grub
Sourcing file '/etc/default/grub'
Sourcing file '/etc/default/grub.d/init-select.cfg'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.8.0-138-generic
Found initrd image: /boot/initrd.img-6.8.0-138-generic
Found linux image: /boot/vmlinuz-6.8.0-111-generic
Found initrd image: /boot/initrd.img-6.8.0-111-generic
Found linux image: /boot/vmlinuz-5.4.282-os2024402055
Found initrd image: /boot/initrd.img-5.4.282-os2024402055
Found memtest86+ image: /boot/memtest86+.elf
Found memtest86+ image: /boot/memtest86+.bin
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
done
os2024402055@kw2024402055:~/26-2_OS/Assignment1/linux-5.4.282$ echo $?
0

```

확인 결과 새로 컴파일한 커널이 GRUB의 부팅 항목에 정상적으로 추가되었음을 확인하였다.

```

os2024402055@kw2024402055:~/26-2_OS/Assignment1/linux-5.4.282$ grep "menuentry '" /boot/grub/grub.cfg | grep 5.4.282
    menuentry 'Ubuntu, with Linux 5.4.282-os2024402055' --class ubuntu --class gnu-linux --class gnu --class os $menuentry_id_option 'gnulinux-5.4.282-os2024402055-advanced-f2b107d8-534c-46ff-9839-2425830c9484' {
        menuentry 'Ubuntu, with Linux 5.4.282-os2024402055 (recovery mode)' --class ubuntu --class gnu-linux --class gnu --class os $menuentry_id_option 'gnulinux-5.4.282-os2024402055-recovery-f2b107d8-534c-46ff-9839-2425830c9484' {

```

vii. Linux Kernel 5.4.282로 부팅

설치한 커널로 부팅하기 위해 GRUB에서 다음 부팅 시 사용할 커널을 지정하였다. 이후 `sudo reboot` 명령어로 시스템을 재부팅하였다.

부팅 완료 후 현재 로그인한 사용자와 실행 중인 커널 버전을 확인했다.

```
os2024402055@kw2024402055: ~  
os2024402055@kw2024402055:~$ whoami  
os2024402055  
os2024402055@kw2024402055:~$ uname -r  
5.4.282-os2024402055  
os2024402055@kw2024402055:~$
```

이를 통해 사용자 계정이 os2024402055로 설정되어 있으며, 직접 컴파일한 Linux Kernel 5.4.282-os2024402055로 정상적으로 부팅되었음을 확인하였다.

viii. 최종 결과

Linux Kernel 5.4.282의 소스 코드를 이용하여 커널을 컴파일하고, EXTRAVERSION을 통해 학번이 포함된 5.4.282-os2024402055 버전을 생성하였다. 커널 및 모듈 설치 후 GRUB에 새 커널을 등록하였으며, 시스템 재부팅 후 `uname -r` 명령어를 실행한 결과 5.4.282-os2024402055가 출력되는 것을 확인하였다. 따라서 Linux Kernel 5.4.282의 컴파일 및 설치가 정상적으로 완료되었다.

C. Assignment 1-3

i. Ctags 및 Cscope 데이터베이스 생성

Linux Kernel 소스 코드에서 함수와 파일의 위치를 효율적으로 탐색하기 위해 Ctags와 Cscope를 사용하였다. 먼저 커널 소스 디렉터리에서 명령어를 실행하여 각 도구에서 사용할 데이터베이스를 생성하였다. 이후 생성된 파일을 확인하였다.

```
os2024402055@kw2024402055:~/26-2_OS/Assignment1/linux-5.4.282$ make tags  
GEN tags  
ctags: Warning: --extra option is obsolete; use --extras instead  
ctags: Warning: --extra option is obsolete; use --extras instead  
ctags: Warning: --extra option is obsolete; use --extras instead  
ctags: Warning: --extra option is obsolete; use --extras instead  
ctags: Warning: --extra option is obsolete; use --extras instead  
ctags: Warning: --extra option is obsolete; use --extras instead  
ctags: Warning: --extra option is obsolete; use --extras instead  
ctags: Warning: --extra option is obsolete; use --extras instead  
ctags: Warning: --extra option is obsolete; use --extras instead  
ctags: Language "kconfig" already defined  
os2024402055@kw2024402055:~/26-2_OS/Assignment1/linux-5.4.282$ echo $?  
0  
os2024402055@kw2024402055:~/26-2_OS/Assignment1/linux-5.4.282$ make cscope  
GEN cscope  
os2024402055@kw2024402055:~/26-2_OS/Assignment1/linux-5.4.282$ echo $?  
0  
os2024402055@kw2024402055:~/26-2_OS/Assignment1/linux-5.4.282$ ls -lh tags cscope.out  
-rw-rw-r-- 1 os2024402055 os2024402055 566M  9월 18 07:52 cscope.out  
-rw-rw-r-- 1 os2024402055 os2024402055 707M  9월 18 07:49 tags
```

실행 결과 tags와 cscope.out 파일이 정상적으로 생성된 것을 확인하였다. 이를 통해 Linux Kernel 전체 소스 코드를 대상으로 Ctags와 Cscope를 활용한 탐색이 가능하도록 준비하였다.

ii. Libux agpgart interface 출력 위치 탐색

Cscope를 실행한 후 Find this text string 기능을 이용하여 "Linux agpgart interface" 문자열을 검색하였다.

```
Text string: Linux agpgart interface

File      Line
0 backend.c 338 printk(KERN_INFO "Linux agpgart interface v%d.%d\n",
```

검색 결과 backend.c의 338번째 줄에서 해당 문자열을 출력하는 printk() 문장을 확인하였다.

```
337         if (!agp_off)
338             printk(KERN_INFO "Linux agpgart interface v%d.%d\n",
339                     AGPGART_VERSION_MAJOR, AGPGART_VERSION_MINOR);
```

이를 통해 Linux agpgart interface 메시지가 drivers/char/agp/backend.c 파일의 agp_init() 함수에서 출력된다는 것을 확인하였다.

iii. Linux agpgart 실행 경로 확인

Ctags를 이용하여 start_kernel() 함수를 탐색하였다. init/main.c의 start_kernel() 함수 마지막 부분에서 다음 코드가 실행되는 것을 확인하였다.

```
761         cpuset_init();
762         cgroup_init();
763         taskstats_init_early();
764         delayacct_init();
765
766         acpi_subsystem_init();
767         arch_post_acpi_subsys_init();
768         sfi_init_late();
769
770         /* Do the rest non-__init'ed, we're now alive */
771         arch_call_rest_init();
772
773         prevent_tail_call_optimization();
774     }
```

arch_call_rest_init()은 다시 rest_init()을 호출한다. 실제 소스에서도 arch_call_rest_init() 내부에서 rest_init()이 호출되고 있으며, start_kernel()의 마지막 단계에서 arch_call_rest_init()이 실행된다.

```
569 void __init __weak arch_call_rest_init(void)
570 {
571     rest_init();
572 }
```

iv. Linux agpgart interface 메시지 수정

첫 번째 요구사항을 만족하기 위해 Linux agpgart interface 메시지를 출력하는 agp_init() 함수를 수정하였다. drivers/char/agp/backend.c 파일에서 기존 메시지 앞에 사용자 계정인 os2024402055를 추가하도록 다음과 같이 수정하였다.

```
static int __init agp_init(void)
{
    if (!agp_off)

        printk(KERN_INFO "os202440255 Linux agpgart interface v%d.%d\n",
                AGPGART_VERSION_MAJOR, AGPGART_VERSION_MINOR);

    return 0;
}

static void __exit agp_exit(void)
{
}
```

KERN_INFO 로그 레벨을 사용하였으며, 기존의

AGPGART_VERSION_MAJOR와 AGPGART_VERSION_MINOR 값을 그대로 출력하도록 하였다.

v. agp_init을 실행하는 함수, 출력

두 번째 요구사항을 구현하기 위해 각각의 initcall 함수를 실제로 실행하는 do_one_initcall()을 수정하였으며, 경로는 `init/main.c`이다.

do_one_initcall() 함수는 initcall_t fn을 전달받아 ret = fn();을 통해 initcall을 실행한다.

fn의 함수명을 문자열로 변환한 후, 해당 함수가 agp_init일 경우에만 메시지를 출력하도록 다음 코드를 추가하였다.

```
snprintf(fn_name, sizeof(fn_name), "%ps", fn);
do_trace_initcall_start(fn);

ret = fn();
if (!strcmp(fn_name, "agp_init"))
    printk(KERN_INFO "os2024402055_arg in do_one_initcall ( %s )\n", fn_name);
```

모든 initcall에서 메시지를 출력하면 동일한 메시지가 여러 번 출력될 수 있으므로, strcmp()를 이용하여 argument로 전달된 함수가 agp_init 일 때만 출력되도록 하였다. 이를 통해 과제에서 요구한 중복 없는 한 세트의 출력이 가능하도록 하였다.

vi. Kernel 컴파일 및 부팅

소스 코드 수정 후 커널을 다시 컴파일하였다.

```
os2024402055@kw2024402055:~/26-2_OS/Assignment1/linux-5.4.282$ make -j4
warning: Cannot use CONFIG_STACK_VALIDATION=y, please install libelf-dev, libelf-devel or elfutils-l
ibelf-devel
CALL scripts/atomic/check-atomics.sh
CALL scripts/checksyscalls.sh
CC init/main.o
CHK include/generated/compile.h
AR init/built-in.a
CHK kernel/kheaders_data.tar.xz
CC drivers/char/agp/backend.o
AR drivers/char/agp/built-in.a
AR drivers/char/built-in.a
OBJCOPY arch/x86/boot/setup.bin
BUILD arch/x86/boot/bzImage
Setup is 17564 bytes (padded to 17920 bytes).
System is 10777 kB
CRC a12cf96a
Kernel: arch/x86/boot/bzImage is ready (#5)
os2024402055@kw2024402055:~/26-2_OS/Assignment1/linux-5.4.282$ echo $?
0
```

컴파일된 커널을 설치한 후 initramfs 및 GRUB 설정을 갱신하고 시스템을 재부팅하였다. 이후 수정한 커널인 5.4.282-os2024402055를 이용하여 정상적으로 부팅하였다.

vii. Kernel Message 출력 결과 확인

수정된 커널로 부팅한 후 dmesg와 grep을 이용하여 추가한 Kernel

Message를 확인하였다. 실행 결과 다음 두 메시지가 연속으로 출력되었다.

```
os2024402055@kw2024402055:~$ sudo dmesg | grep -E "os2024402055_(Linux agpgart|arg)"
[sudo] password for os2024402055:
[ 13.069114] os2024402055_Linux agpgart interface v0.103
[ 13.069115] os2024402055_arg in do_one_initcall ( agp_init )
```

첫 번째 메시지를 통해 agp_init()에서 Linux agpgart interface 버전 0.103이 정상적으로 출력됨을 확인하였다. 두 번째 메시지를 통해 do_one_initcall()의 argument로 agp_init이 전달되어 실제로 실행되었음을 확인하였다.

또한 두 메시지의 출력 시간이 거의 동일하며 서로 연속해서 출력되고, 동일 메시지가 반복되지 않는 것을 확인하였다. 따라서 Linux agpgart가 실행되는 지점에서 요구된 두 Kernel Message가 한 세트만 출력되도록 정상적으로 구현되었다.

viii. 최종 결과

Ctags와 Cscope를 활용하여 Linux Kernel 소스 코드에서 Linux agpgart interface가 출력되는 위치와 관련 함수들을 탐색하였다.

drivers/char/agp/backend.c의 agp_init()에서 Linux agpgart interface 메시지를 수정하였으며, init/main.c의 do_one_initcall()에서 argument로 전달된 함수명을 확인하여 agp_init일 경우에만 추가 Kernel Message가 출력되도록 구현하였다.

수정된 커널을 컴파일 및 설치한 후 시스템을 재부팅하여 dmesg와 grep으로 확인한 결과, os2024402055_Linux agpgart interface v0.103과 os2024402055_arg in do_one_initcall (agp_init)가 연속해서 한 번씩 출력되었다. 따라서 Linux agpgart 실행 위치에서 Kernel Message를 출력하고, 해당 함수를 실행시키는 함수명과 argument 값을 출력하는 Assignment 1-3의 요구사항을 만족하였다.

3. 고찰

이번 과제를 통해 Linux Kernel의 소스 코드를 직접 컴파일하고, Ctags와 Cscope를 활용하여 원하는 함수와 실행 위치를 탐색하는 과정을 경험할 수 있었다. 특히 커널 컴파일 과정에서 EXTRAVERSION 뒤의 공백으로 인해 모듈 설치 경로가 잘못 생성되는 문제와, initramfs 문제로 커널 부팅 시 Kernel Panic이 발생하는 문제를 겪었다. 이를 통해 커널 빌드에서는 작은 설정 하나도 전체 실행 과정에 큰 영향을 줄 수 있다는 점을 알 수 있었다.

또한 start_kernel()에서 시작하여 do_one_initcall()을 통해 agp_init()이 실행되는 흐름을 따라가면서, 커널 초기화 함수들이 어떤 방식으로 호출되는지 이해할 수

있었다. 최종적으로 `printk()`를 이용해 원하는 Kernel Message를 출력하고 `dmesg`로 결과를 확인하면서, 커널 소스 수정부터 실제 부팅 결과 확인까지의 전체 과정을 직접 수행할 수 있었다.

4. Reference

- A. 광운대학교, 「26_2_HAI_OS_Assignment1_v2.pdf」, Operating System Assignment #1 과제 자료, 2026.
- B. The Linux Kernel Archives, Linux Kernel 5.4.282,
<https://cdn.kernel.org/pub/linux/kernel/v5.x/linux-5.4.282.tar.xz>
- C. Linux Kernel 5.4.282 Source Code, `init/main.c`, `drivers/char/agp/backend.c`